



**Department of Electrical,
Computer, & Biomedical Engineering**
Faculty of Engineering
& Architectural Science

Course Title:	COE
Course Number:	328
Semester/Year:	F2024

Instructor:	Dr. Vadim Geurkov
--------------------	-------------------

<i>Assignment Number/Lab Number:</i>	Lab 6
<i>Assignment/Lab Title:</i>	Design of a Simple Central Processing Unit

<i>Submission Date:</i>	12 December 2024
<i>Due Date:</i>	12 December 2024

Student LAST Name	Student FIRST Name	Student Number	Section	Signature*
Phan	Henry	501232910	06	<i>Henry</i>
Nguyen	Jack	501241070	06	<i>Jack</i>

*By signing above you attest that you have contributed to this written lab report and confirm that all work you have contributed to this lab report is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a "0" on the work, an "F" in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at: <https://www.torontomu.ca/content/dam/senate/policies/pol60.pdf>

1.0 TABLE OF CONTENTS

<i>2.0 - INTRODUCTION.....</i>	<i>3</i>
<i>3.0 - COMPONENTS.....</i>	<i>3</i>
<i>3.1 - D FLIP-FLOP.....</i>	<i>3</i>
<i>3.2 - 3 TO 8 DECODER.....</i>	<i>5</i>
<i>3.3 - FSM.....</i>	<i>6</i>
<i>4.0 - ALU_1.....</i>	<i>8</i>
<i>5.0 - ALU_2 (Problem B).....</i>	<i>10</i>
<i>6.0 - ALU_3 (Problem B).....</i>	<i>13</i>
<i>7.0 - CONCLUSION.....</i>	<i>14</i>
<i>REFERENCES.....</i>	<i>15</i>

2.0 - INTRODUCTION

In this lab, with the accumulation of knowledge throughout the course, the individual components previously created in other labs will now be combined together to create a simple control processing unit (CPU). The design of this CPU incorporates storage elements, display elements for the output, combinational logic elements, sequential logic elements, and a control unit which was made up of an FSM and 3:8 decoder. With all these elements working in unison, multiple functions can be implemented for the CPU to produce various outputs depending on the corresponding function tested. These sections will be explained more in depth throughout the report which include: components, ALU_1, ALU_2, and ALU_3.

3.0 - COMPONENTS

In this section, the main components that will be introduced are latches, 3:8 decoder, and FSM. These components each play a crucial role, which will then be combined together to create a simple CPU. Each component section will consist of their description, truth table, block diagrams, and waveform.

3.1 - D FLIP-FLOP

The D flip-flop is a storage device that can hold one bit of data. The synchronous D flip-flop takes in one data input and stores it on the positive edge of the clock input. There can also be a 'clear' input that instantly sets the stored data to zero when it is high. In total, sixteen synchronous D flip-flops with a clear input were split into two registers to store the 8-bit inputs, A and B.

8-bit Register using Synchronous D flip-flops:

Clock (clk)	Reset (res)	A[7...0]	Q[7...0]
^	1	00000000	00000000
^	1	11110000	00000000
^	0	00000000	00000000
^	0	10011010	10011010
^	0	11111111	11111111

Table 1: This is the truth table for an 8-bit register using D flip-flops

Register A Block Symbol Diagram:

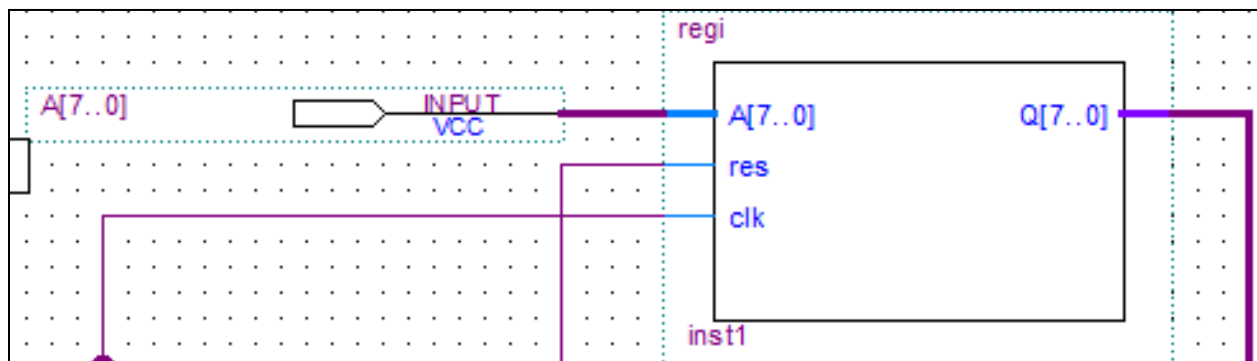


Figure 1: This figure represents the block diagram for register of A

Register B Block Symbol Diagram:

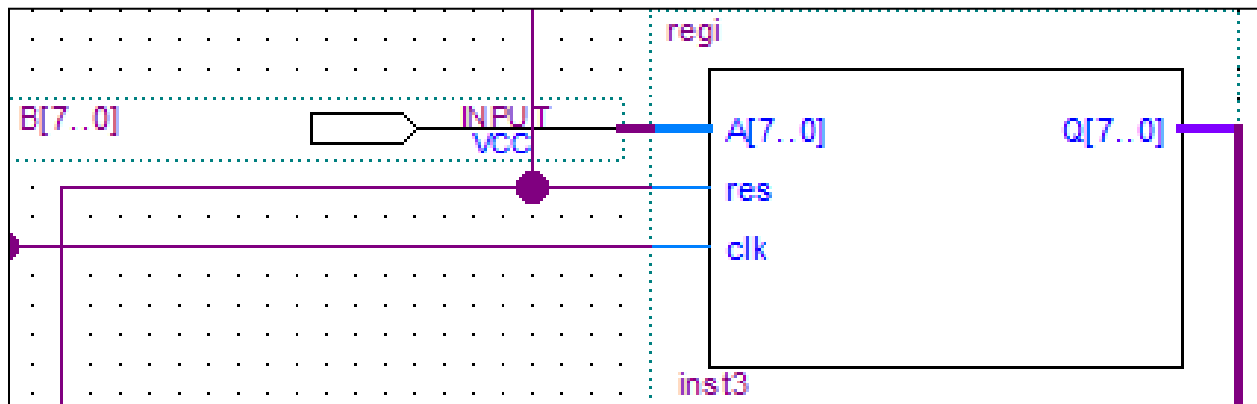


Figure 2: This figure represents the block diagram for register of B

3.2 - 3 TO 8 DECODER

The 3 to 8 decoder is a component that takes in encoded information, and decodes it. The way this decoder will work is that it will take in a 3-bit input and depending on the input, the decoder will select one of the eight 8-bit outputs to signal as the output. With this 3-bit input, it is accompanied with the enable input, which determines when the decoder is enabled. In other words, it will disable its inputs if the enable input disables the decoder, and it will operate normally when the decoder is enabled. The block diagram for this 3 to 8 decoder is shown below in **Figure 3**.

3 to 8 Decoder Truth Table:

Enable (En)	Inputs (w[2..0])	Outputs (y[7...0])
0	xxx	00000000
1	000	00000001
1	001	00000010
1	010	00000100
1	011	00001000
1	100	00010000
1	101	00100000
1	110	01000000
1	111	10000000

Table 2: This is the truth table for a 3 to 8 decoder

3 to 8 Decoder Block Symbol Diagram:

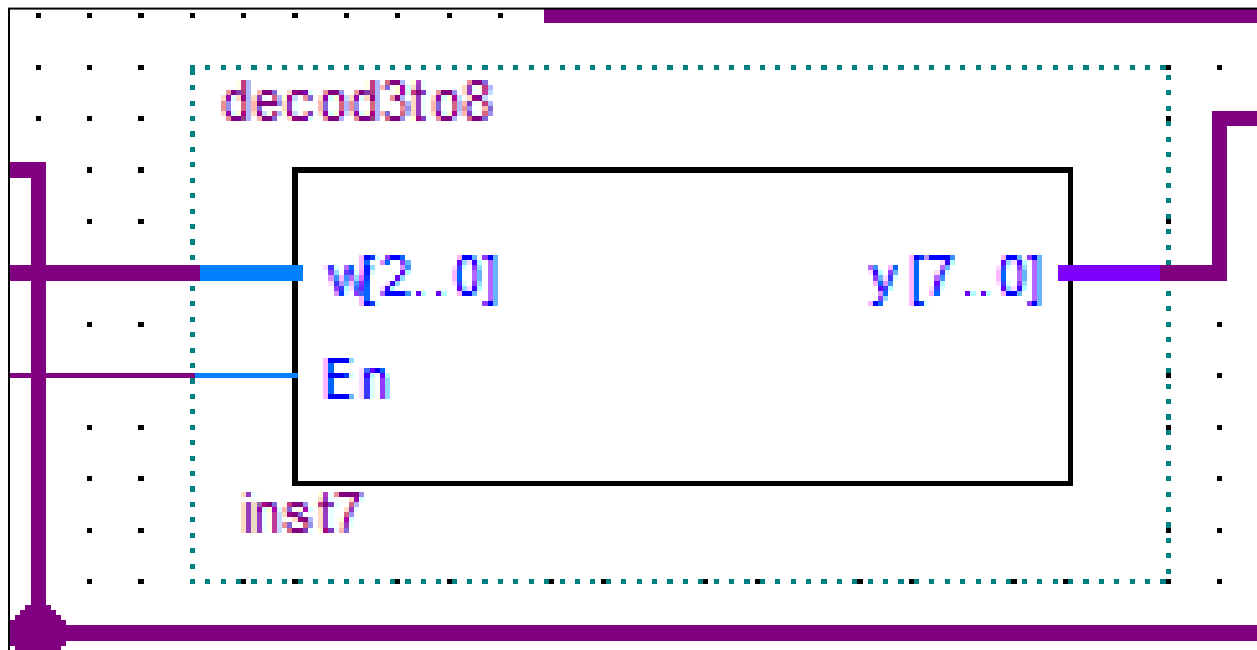


Figure 3: This figure represents the block diagram for the 3 to 8 decoder

3.3 - FSM

The Finite State Machine (FSM) is a component which takes in a current state and an input to control its next state which will produce an output depending on the next state. There are two different types of FSMs (mealy and moore), but for this project, the moore FSM was implemented. The significance of the moore FSM is that it only changes the state depending on the current state it is in and it does not directly depend on the input, unlike the mealy FSM where it depends on the input. For this circuit, the inputs are the clock, reset, and `data_in`, and the outputs are the `student_id` and `current_state` as shown in **Figure 4**.

FSM Truth Table:

*Note: The student ID used for this truth table was 501232910 (Henry Phan)

Current State	Next State (data_in)		Output (student_id[3..0])
	When x = 0	When x = 1	
0000	0000	0001	0000
0001	0001	0010	0001
0010	0010	0011	0010
0011	0011	0100	0011
0100	0100	0101	0010
0101	0101	0110	1001
0110	0110	0111	0001
0111	0111	0000	0000

Table 3: This is the truth table for the Moore FSM

FSM Block Symbol Diagram:

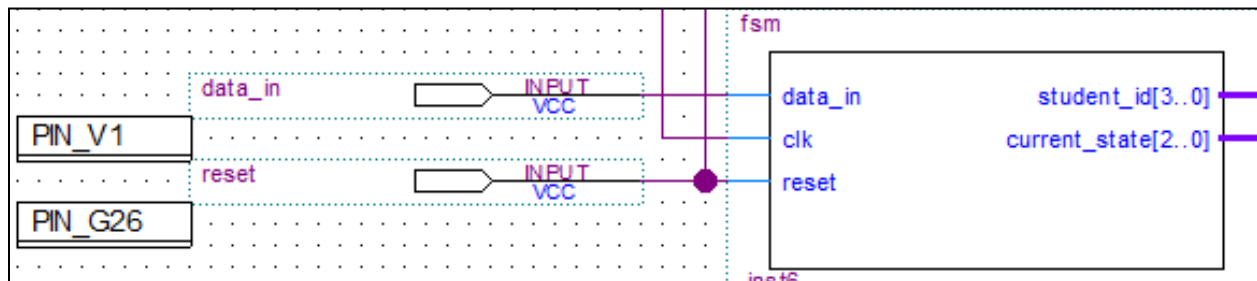


Figure 4: This is block diagram for the Moore FSM

4.0 - ALU_1

The purpose of ALU_1 was to perform a function depending on the opcode selected. It will take the two values from each register and it will use those values as the input for the ALU. Then, the result from performing the function will produce an output result that will be stored in another register. This value will then be sent to the four 7 segment display to put onto the altera board. This example of how the ALU is implemented can be seen in **Figure 5**.

Table of Functions:

Function #	Opcode	Function
1	00000001	$sum(A, B)$
2	00000010	$dif(A, B)$
3	00000100	$\overline{A}$
4	00001000	$\overline{A \cdot B}$
5	00010000	$\overline{A + B}$
6	00100000	$A \cdot B$
7	01000000	$A \oplus B$
8	10000000	$A + B$

Table 4: This table represents the ALU operations for Problem 1 [1]

ALU_1 Block Diagram:

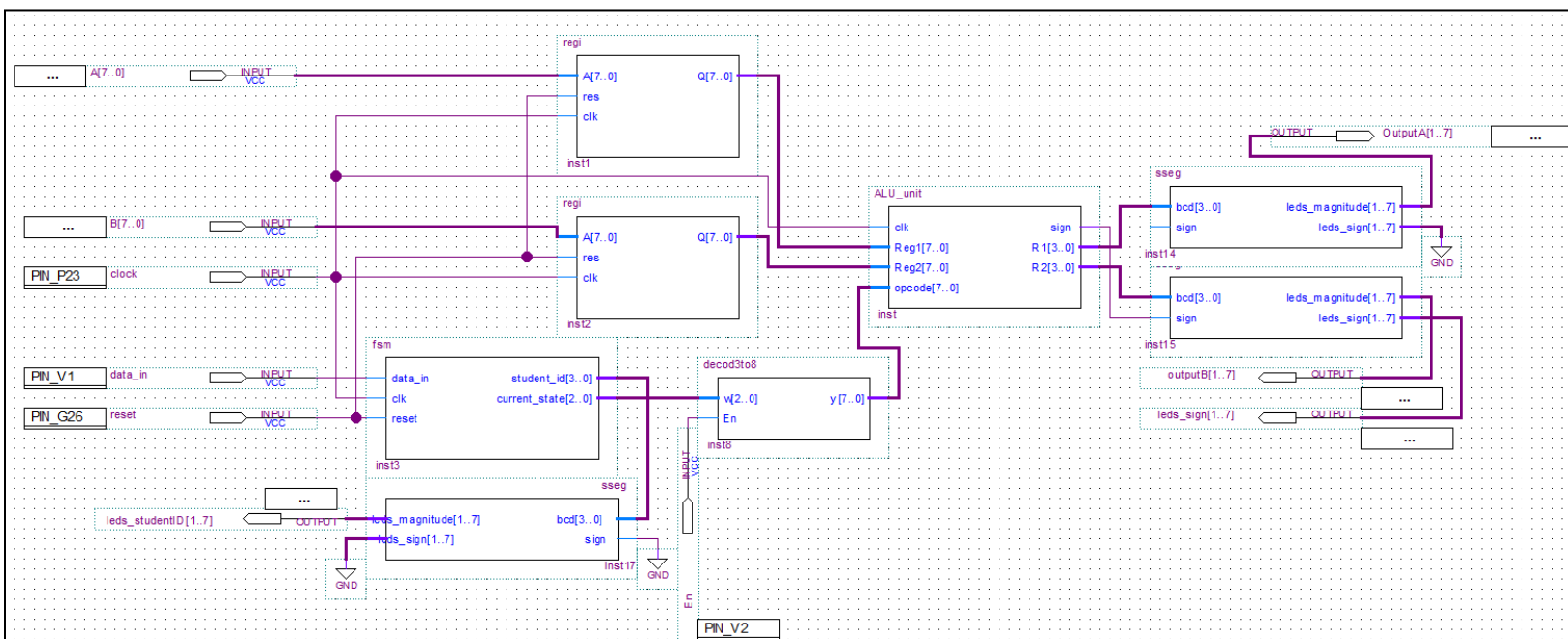


Figure 5: This is the Block Diagram for the ALU in Problem 1

ALU_1 Inputs:

- Clock
 - This input corresponds to the clock signal that keeps all of the components synchronized.
- Opcode
 - This input corresponds to the function that will be selected to execute.
- Reg1
 - This input corresponds to the 8-bit binary value (A) inputted for this register block.
- Reg2
 - This input corresponds to the 8-bit binary value (B) inputted for this register block.

ALU_1 Outputs:

- R1
 - This output corresponds to the last 4 bits of the 8-bit result from performing the function.
- R2
 - This output corresponds to the first 4 bits of the 8-bit result from performing the function.
- Sign
 - This output corresponds to the sign of the result.

5.0 - ALU_2 (Problem B)

Similarly to ALU_1, the purpose of ALU_2 was to perform a function depending on the opcode selected. It will take the two values from each register and it will use those values as the input for the ALU. Then, the result from performing the function will produce an output result that will be stored in another register. This value will then be sent to the four 7 segment display to put onto the altera board. This example of how the ALU is implemented can be seen in **Figure 6**.

Table of Functions:

Function #	Opcode	Operation
1	00000001	Swap the lower and upper 4 bits of A
2	00000010	Produce the result of ORing A and B
3	00000100	Decrement B by 5
4	00001000	Invert all bits of A
5	00010000	Invert the bit-significance order of A
6	00100000	Find the greater value of A and B and produce the results (Max(A, B))
7	01000000	Produce the difference between A and B
8	10000000	Produce the result of XNORing A and B

Table 5: This table represents the ALU operations for Problem 2b [1]

ALU_2 Block Diagram:

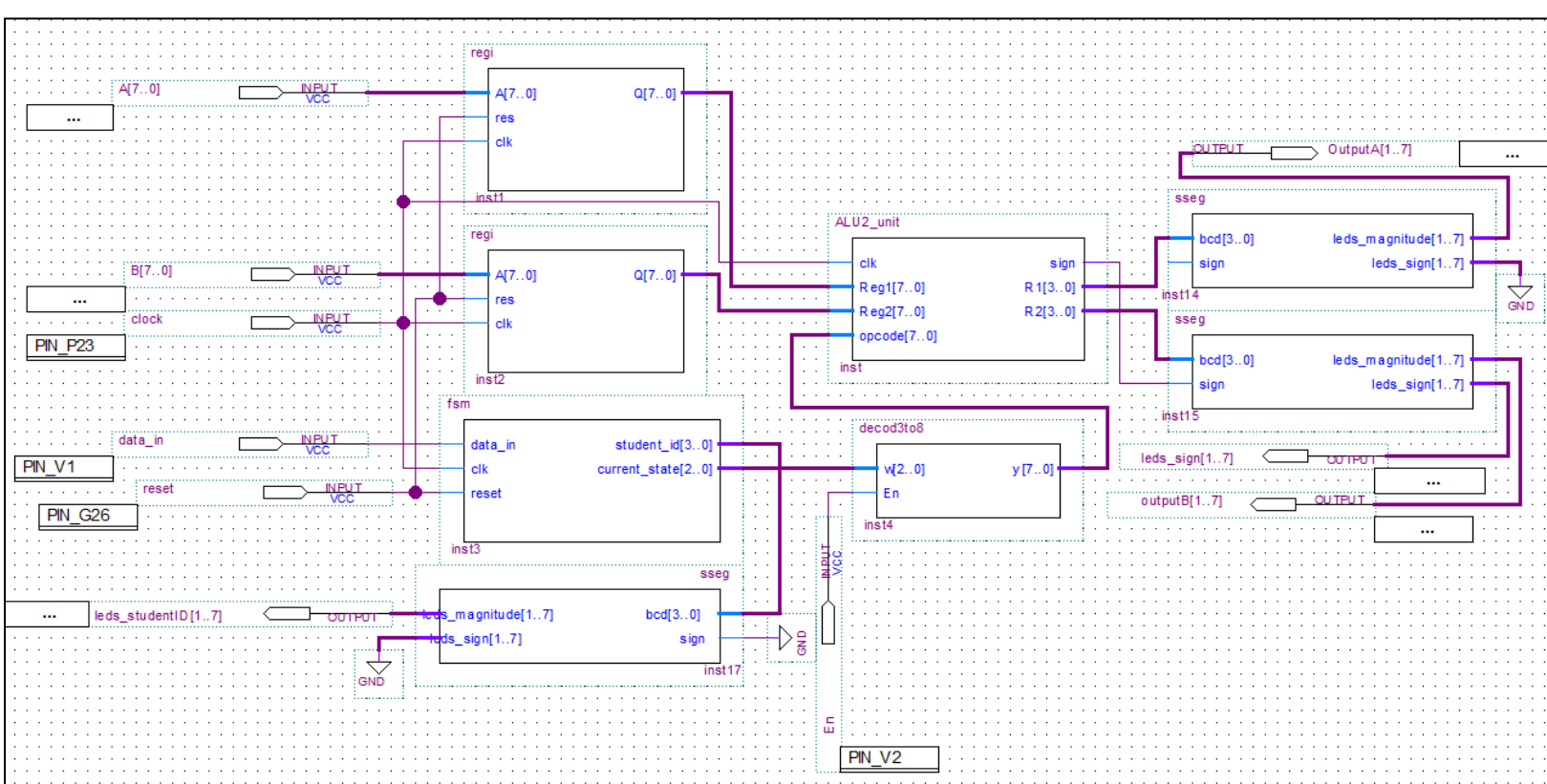


Figure 6: This is the block diagram for the ALU in Problem 2

ALU_2 Inputs:

- Clock
 - This input corresponds to the clock signal that keeps all of the components synchronized.
- Opcode
 - This input corresponds to the function that will be selected to execute.
- Reg1
 - This input corresponds to the 8-bit binary value (A) inputted for this register block.
- Reg2
 - This input corresponds to the 8-bit binary value (B) inputted for this register block.

ALU_2 Outputs:

- R1
 - This output corresponds to the last 4 bits of the 8-bit result from performing the function.
- R2
 - This output corresponds to the first 4 bits of the 8-bit result from performing the function.
- Sign
 - This output corresponds to the sign of the result.

6.0 - ALU_3 (Problem B)

The purpose of ALU_3 was to show if a binary number given from the input student_id was even or odd. On the positive edge, the ALU outputs 0000 when the input is even and 0001 when the input is odd. This was done by checking if the LSB of the input student_id was high or low. If the LSB was high, the input was not even, if the LSB was low, the input was even.

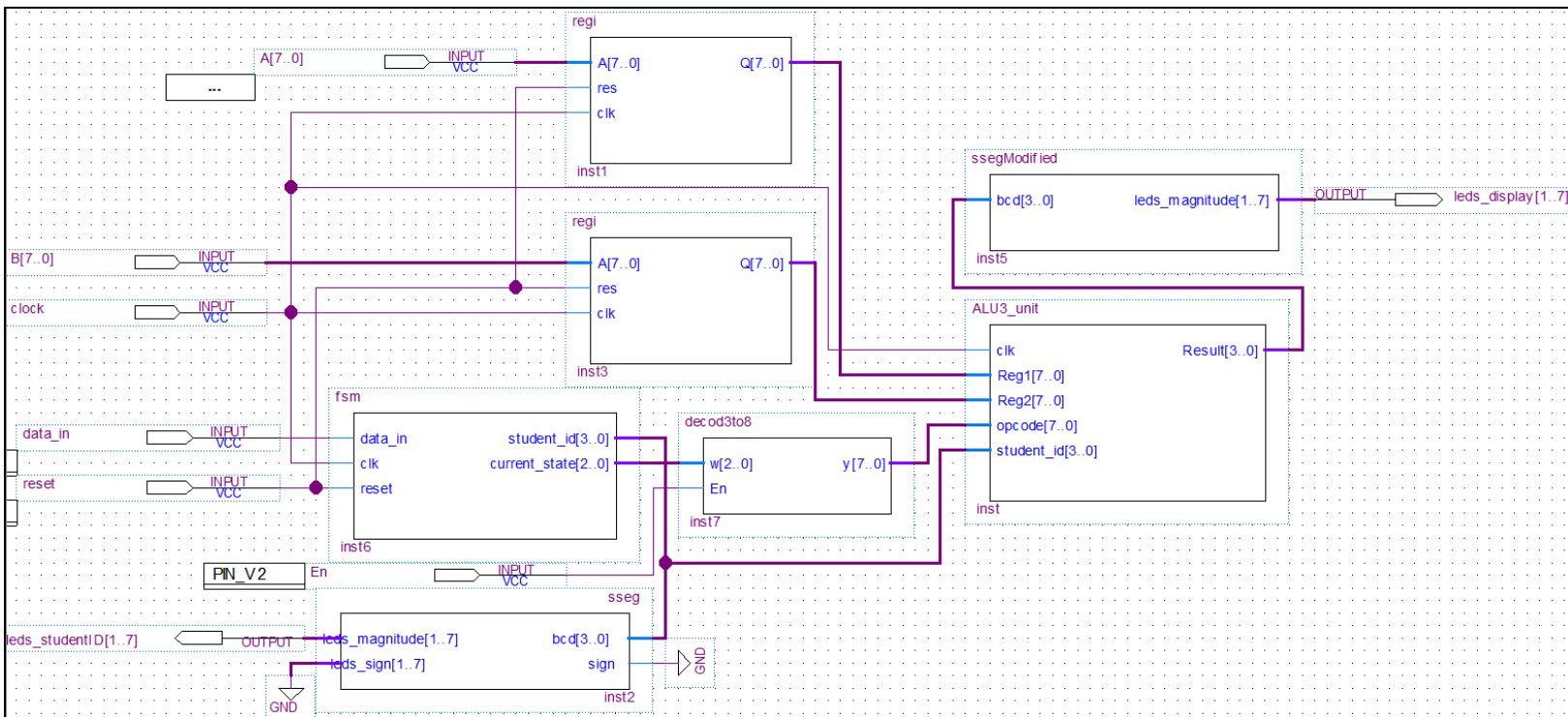
ALU_3 Block Diagram:

Figure 7: This is the block diagram for the ALU in Problem 3

ALU_3 Inputs:

- clk
 - This input corresponds to the clock signal that keeps all of the components synchronized.
- Reg1
 - This input has no impact on the output
- Reg2
 - This input has no impact on the output
- Opcode
 - This input has no impact on the output
- student_id *Note: The student ID used was 501232910 (Henry Phan)
 - This input corresponds to the 4-bit binary value given by the FSM.

ALU_3 Outputs:

- Result
 - This output corresponds to the 4-bit binary for even or not even.
 - 0000 → Even (Y)
 - 0001 → Not even (N)

7.0 - CONCLUSION

In conclusion, we learned about delay in synchronized components, the function of a register, the process of creating an ALU, and how to implement functions. By creating and testing waveforms, we learned about how a component that is positive-edge-triggered has a delay. The process of creating the ALU and implementing its functions gave us the opportunity to solve problems such as, inverting the bit-significance order in VHDL. Overall, many lessons were learned and many problems were solved in this successful experiment.

REFERENCES

- [1] Toronto Metropolitan University. (n.d.). Lab 6 - Design of a Simple Central Processing Unit.
courses.torontomu.ca. Retrieved December 12, 2024, from
<https://courses.torontomu.ca/d2l/le/content/909747/viewContent/5997374/View>